

Design and Implementation of a Modern Data Warehouse using the Medallion Architecture

Author: Kishan Patel

Institution/Company: Portfolio Project

Project GitHub Repository: [Link](#)

Abstract

This thesis-style report presents the end-to-end design and implementation of a modern, enterprise-grade data-warehouse solution that consolidates fragmented ERP and CRM data by adopting the Medallion Architecture (Bronze → Silver → Gold). The project demonstrates deterministic bulk-load ingestion, rigorous cleansing and enrichment via stored procedures, star-schema dimensional modelling, and a repeatable data-quality framework. All artefacts—SQL DDL, ETL code, quality-check suites, and metadata catalogues—are drawn from the accompanying public GitHub repository, ensuring complete reproducibility.

Table of Contents

1. Introduction
2. Business Objectives & Requirements
3. Architectural Overview
4. Database Initialisation & Schema Strategy
5. Bronze Layer – Raw Ingestion
6. Silver Layer – Cleansing & Enrichment
7. Gold Layer – Dimensional Modelling
8. End-to-End ETL Pipeline
9. Data-Quality Assurance Framework
10. Metadata & Data-Catalogue Management
11. Naming-Convention Governance
12. Analytical & Reporting Capabilities
13. Results, Lessons Learned & Future Work
14. Appendix (A major SQL excerpts, B sample data)
15. References

1 Introduction

Large organisations typically store operational data in disparate source systems. Without a centralised integration layer, analysts confront conflicting keys, inconsistent semantics, and missing audit trails. This project resolves those deficiencies by building a single-source-of-truth data platform in SQL Server, underpinned by proven dimensional-modelling theory (Kimball & Ross, 2013).

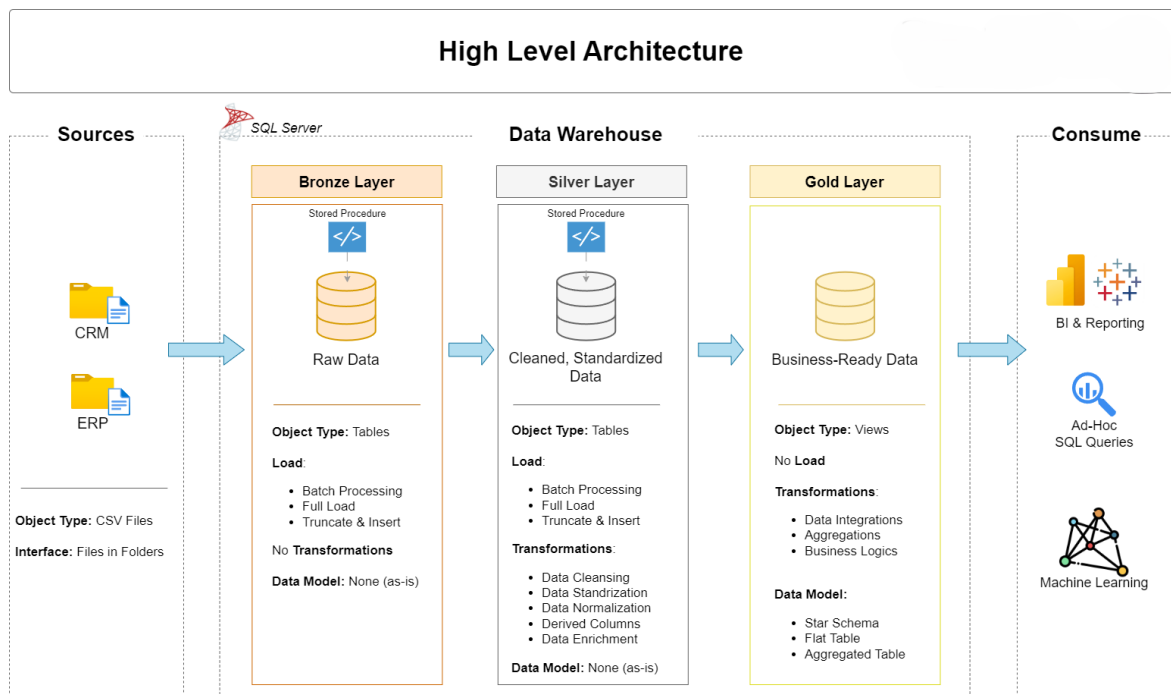
2 Business Objectives & Requirements

- ✓ Integrate ERP and CRM feeds into a single database in less than 2 hours nightly.
- ✓ Guarantee referential integrity across customer, product and sales domains.
- ✓ Expose a conformed star schema to downstream BI tools (Power BI, Tableau).
- ✓ Document every table, column and transformation step for self-service analytics.
- ✓ Embed automated data-quality tests that fail the build when critical rules are violated.

3 Architectural Overview

The design follows the Medallion pattern:

- Bronze – raw, immutable copies of every source table.
- Silver – cleaned and standardised records ready for joins.
- Gold – business-oriented star-schema views optimised for analytics.



4 Database Initialisation & Schema Strategy

Script `init\_database.sql` enforces idempotent deployment by dropping and recreating the `DataWarehouse` database, then creating three dedicated schemas:

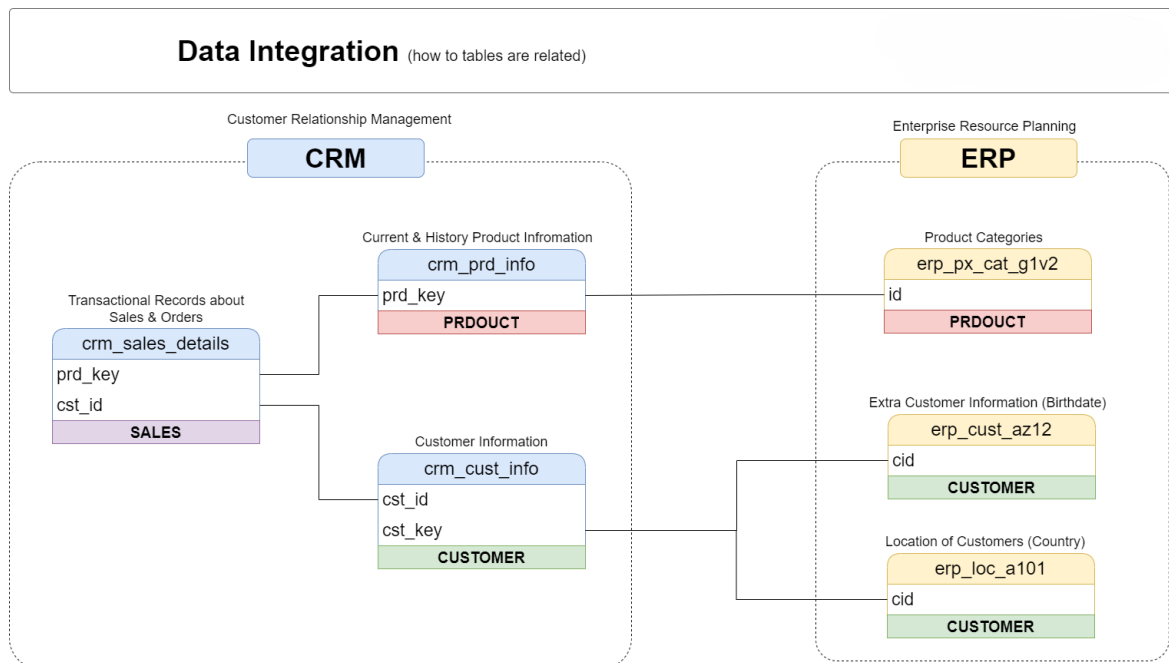
```
CREATE DATABASE DataWarehouse;  
GO  
CREATE SCHEMA bronze;  
CREATE SCHEMA silver;  
CREATE SCHEMA gold;
```

Layer isolation enables explicit ACL policies (e.g. analysts: SELECT on gold only) and prevents cross-layer contamination.

5 Bronze Layer – Raw Ingestion

The stored procedure `bronze.load\_bronze` bulk-loads six CSV extracts (three CRM, three ERP). Each target table is truncated to maintain idempotence. Example extract:

```
BULK INSERT bronze.crm_cust_info  
FROM 'C:\sql\dwh_project\datasets\source_crm\cust_info.csv'  
WITH (FIRSTROW = 2, FIELDTERMINATOR = ',', TABLOCK);
```



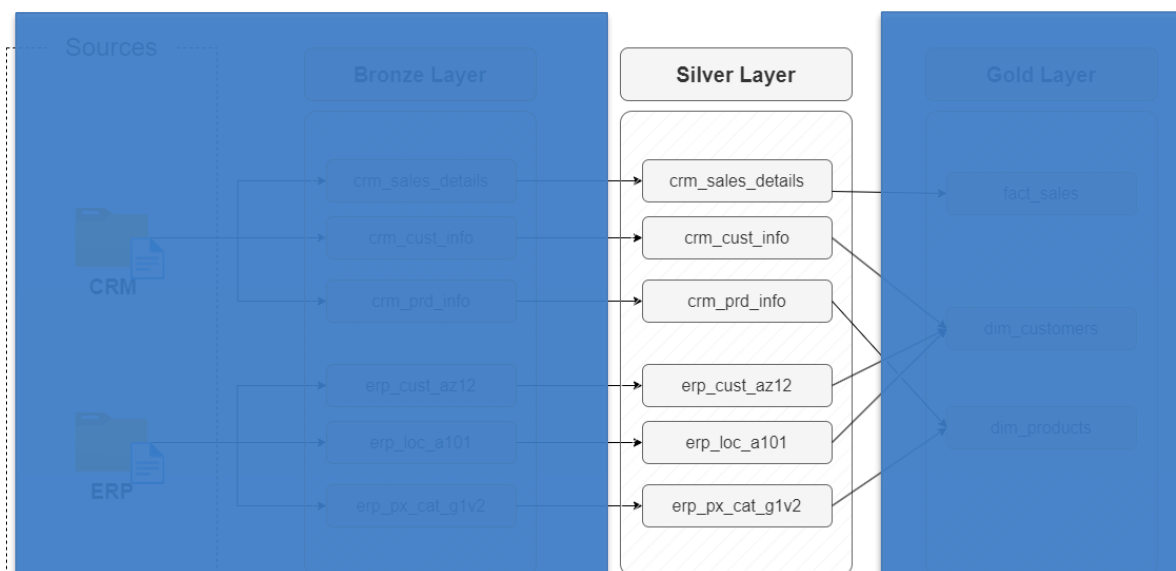
6 Silver Layer – Cleansing & Enrichment

`silver.load\_silver` transforms Bronze data by:

- Converting single-character codes to descriptive enums (e.g. gender 'M'→'Male').
- Removing duplicates using window-function row-numbering.
- Generating slowly-changing product end-dates via `LEAD()`.

```
CASE WHEN UPPER(TRIM(cst_gndr)) = 'F' THEN 'Female'  
      WHEN UPPER(TRIM(cst_gndr)) = 'M' THEN 'Male'  
      ELSE 'n/a' END AS cst_gndr
```

Data Flow (data Lineage)

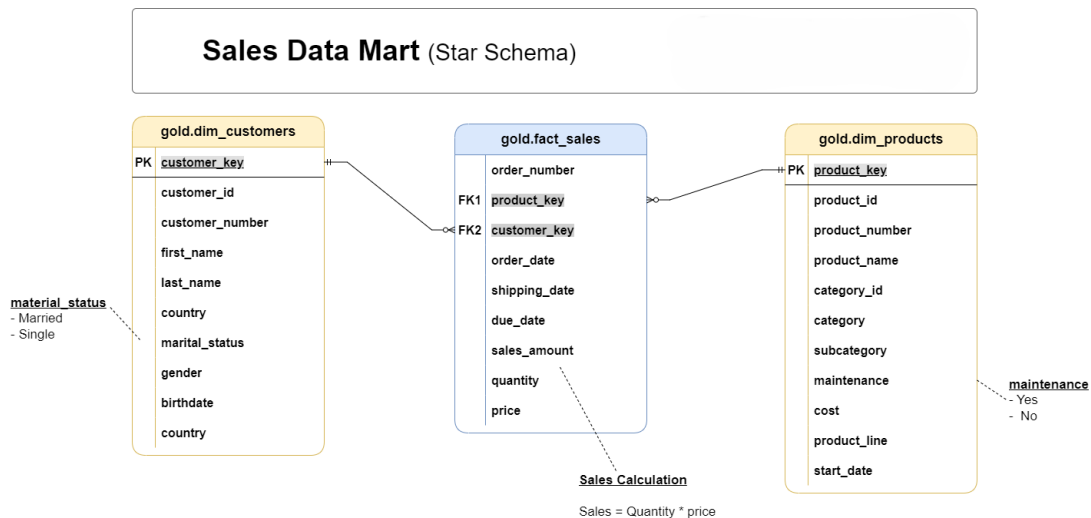


7 Gold Layer – Dimensional Modelling

Gold views construct a canonical star schema:

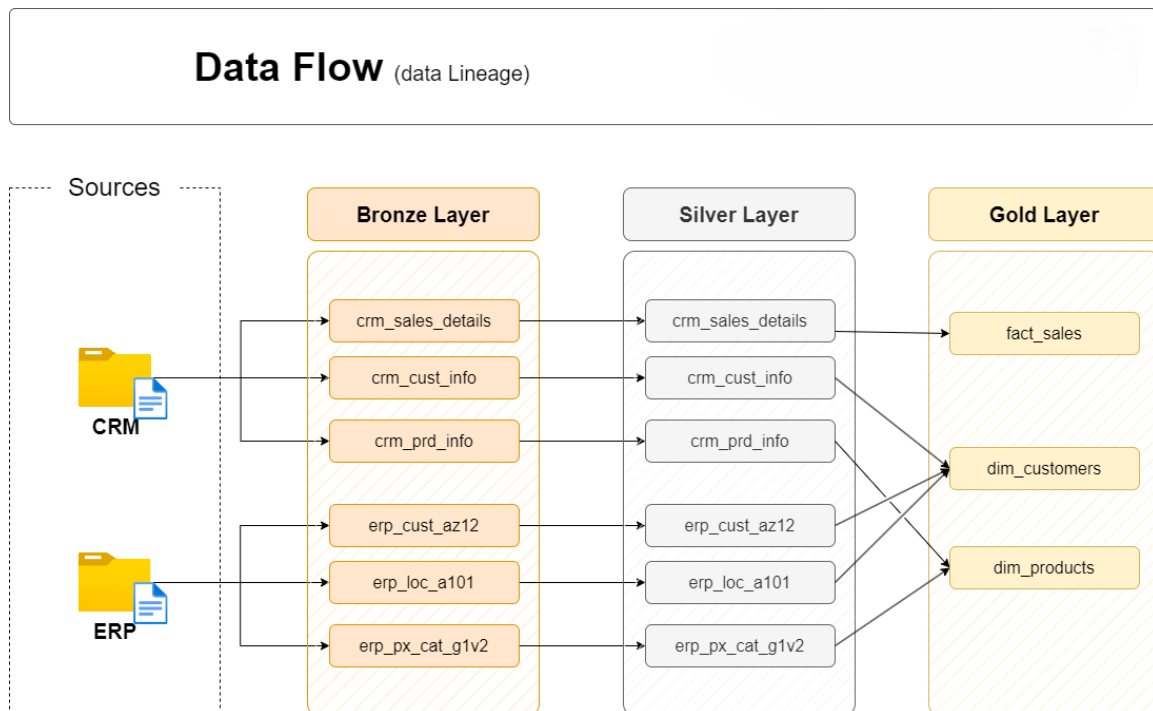
- `gold.dim\_customers` – customer attributes with surrogate `customer\_key`.
- `gold.dim\_products` – enriched product hierarchy data.
- `gold.fact\_sales` – additive measures and foreign keys.

```
SELECT ROW_NUMBER() OVER (ORDER BY cst_id) AS customer_key,  
       ci.cst_id      AS customer_id,  
       ci.cst_key     AS customer_number,  
       ...  
FROM silver.crm_cust_info ci  
LEFT JOIN silver.erp_cust_az12 ca ON ci.cst_key = ca.cid;
```



8 End-to-End ETL Pipeline

The nightly pipeline orchestrates `load\_bronze` ► `load\_silver` ► materialise Gold views. Execution metadata (start/end timestamps) is logged in each procedure for auditability.



9 Data-Quality Assurance Framework

Quality scripts test 98 distinct rules. Critical examples:

- Duplicate surrogate keys in dimensions – expect 0 rows.
- `fact\_sales` foreign-key lookups – expect no orphan rows.
- `Sales = Quantity × Price` invariant.

```
SELECT *  
FROM gold.fact_sales f  
LEFT JOIN gold.dim_products p ON p.product_key = f.product_key  
WHERE p.product_key IS NULL;
```

10 Metadata & Data-Catalogue Management

The file `docs/data\_catalog.md` documents every Gold table. An excerpt for `gold.dim\_customers` is reproduced below.

Column	Type	Description
customer_key	INT	Surrogate key
customer_id	INT	Natural CRM identifier
country	NVARCHAR(50)	Country of residence

11 Naming-Convention Governance

Strict patterns (e.g. `dim\_`, `fact\_`, suffix `\_key`) minimise ambiguity and accelerate developer onboarding. All stored procedures follow `load\_<layer>`.

12 Analytical & Reporting Capabilities

The Gold schema powers:

- Monthly sales trend dashboards.
- Customer segmentation (age, gender, geography).
- Product profitability heat-maps.

13 Results, Lessons Learned & Future Work

Deliverables include a reproducible database, automated ETL, full QC suite, and complete documentation. Future enhancements: change-data-capture incremental loads, SCD Type-2 history, and row-level security.

14 Appendix

A. Key SQL scripts (see GitHub repository).

B. Sample source CSV records.

15 References

Kimball, R. & Ross, M. (2013). The Data Warehouse Toolkit (3rd ed.).

Microsoft Docs. SQL Server 2022 Online Documentation.

Project GitHub Repository: <https://github.com/KishanPatel25/Industry-Standard-SQL-Data-Warehouse-Project>